

Deep Learning for Soybean Phenotype Prediction: A Convolutional Neural Network Based Approach

Suleman Khan
skhan94@uoguelph.ca
University of Guelph
Guelph, Ontario, Canada

Abstract

Soybean plays a vital role in global food security, motivating advances in genomic selection for improved breeding outcomes. Traditional statistical models rely on strong prior assumptions and often require genetic imputation to address missing genotype data. This study investigates deep learning approaches, particularly convolutional neural networks (CNNs), as an alternative framework for phenotype prediction. This study compares the performance of CNN-based models (deepGS and Dual CNN) with classical statistical baselines across both imputed and non-imputed datasets from the SoyNAM panel. Results reveal that deep learning models not only match the accuracy of statistical methods but also exhibit robustness to missing genotype data, mitigating the need for imputation. Dual CNN achieves the highest prediction accuracy with faster convergence due to its parallel architecture, highlighting the importance of architectural design in phenotype prediction tasks. This study demonstrates the efficiency of deep models in plant genomics and underscores their potential to transform phenotype prediction pipelines in breeding programs.

Keywords

Deep Learning, Genomic Selection, Phenotype Prediction, Convolutional Neural Network

ACM Reference Format:

Suleman Khan. 2025. Deep Learning for Soybean Phenotype Prediction: A Convolutional Neural Network Based Approach. In *Proceedings of This work is done during graduate course at the University of Guelph. (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXX.XXXXXXX>

1 Introduction

Soybean is among the most important dicotyledons, playing a crucial role in global nutrition [7]. To meet global demand, researchers try to enhance soybean crop breeding. Traditional breeding methods, such as mass selection and pure line selection, to enhance crop breeding have several limitations [2], [6]. Marker assisted selection (MAS) improves phenotype prediction for quantitative variables in breeding. This method assumes genotype markers have strong

relationships with phenotypes. Genomic selection (GS) is a MAS method that uses single nucleotide polymorphisms (SNPs) to predict phenotypes. This strategy has been widely applied for major crops such as rice, wheat, and Holstein dairy cattle.

Traditional statistical methods such as best linear unbiased prediction (BLUP) and Bayesian lasso regression for phenotype prediction have limitations. These methods depend on prior assumptions. Bayesian models assume prior distribution of parameters and likelihood function validity [16]. These models consider the contribution of each genotype to phenotype as an independent feature. However, in practice, this genotype effect is unknown and SNPs interact with other SNPs that contribute to complex traits in organisms. [19].

Another challenge for statistical models is missing values in the genotype data matrix. These missing values screened out during data processing stage through genetic imputation [18]. In genetics, imputation is the statistical inference of unobserved genotypes. It is achieved by using known population template [7]. Several genetic imputation methods have been developed that work with or without reference genomes. Some imputation methods are expectation-maximization (EM) algorithm with decision tree [14], calculated mean, random forest in missForest [17], hidden Markov model in BEAGLE [13], and MaCH [8]. For phenotype prediction through a statistical model, genotype data is first imputed, then divided into training and testing sets. To some extent, the training set is not entirely independent from the testing set in this case and can affect the accuracy of phenotype prediction. Imputation is dependent on non-missing values of genotype, so it can introduce noise in the dataset.

In recent years, deep learning has been applied in computational biology. It also has successful applications in genomic studies, such as gene expression levels prediction [15] and functional genomic elements identification [12]. Deep learning models, such as DeepGS [10] and rrBLUP [1] have been used to predict phenotypes using non imputed SNPs. A simple dense neural network (DNN) [11] used genotype-by-sequencing (GBS) data, and Dual CNN [9] used parallel convolutional architecture, improved phenotype prediction using deep learning.

Convolutional neural networks use a grid like structure and can capture spatial information from genomic data. Convolutional layers in CNNs have the ability to extract feature maps from raw genomic data. CNN can also resolve gene-gene interaction effect also called epistasis effect. The convolutional layer considers those interactions between SNPs with different kernel window sizes.

In this study, I used the CNN models for phenotype prediction considering their various advantages in previous genomic studies and compared them with statistical models. This study explored two research questions.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted by ACM, provided that the copies are not made for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2025/04

<https://doi.org/XXXXXX.XXXXXXX>

RQ1 : Which models achieve better performance when comparing deep learning models to statistical models for soybean phenotype prediction ?

RQ2 : How can genetic imputation affect the performance of soybean phenotype prediction models ?

CNN models can use both imputed and non imputed data, while statistical models used imputed data only for phenotype prediction. Therefore, the effect of imputed data is studied on CNN models only. In addition, the computational complexity of each model is studied to draw inference about the design and architecture of phenotype prediction models.

2 Methods

2.1 Problem formulation

Let $\mathcal{D} = \{(X_i, y_i)\}_{i=1}^N$ be the dataset, where:

- $X_i \in \mathbb{R}^d$ represents the **genotypic features** (e.g., SNP markers) for the i -th sample.
- $y_i \in \mathbb{R}$ represents the **phenotypic traits** (e.g., protein, height, oil).
- N is the total number of samples.
- d is the number of genomic features (SNPs).

genotypic data X **imputed** to handle missing values, denoted as X^{imp} .

Given dataset $\mathcal{D}$, how do deep learning models f_θ and statistical models g_ϕ perform phenotype prediction in imputed and non-imputed settings?

$$\hat{y} = f_\theta(X) \quad \text{vs} \quad \hat{y} = g_\phi(X). \quad (1)$$

$$\hat{y}^{\text{imp}} = f_\theta(X^{\text{imp}}) \quad \text{vs} \quad \hat{y} = f_\theta(X). \quad (2)$$

To assess computational efficiency, this study analyzed the complexity of each model in terms of **training time t** , measured as the number of iterations until convergence.

2.2 Dual-stream Convolutional Neural Network

Liu et al. [9] dual-stream convolutional neural network based on three building blocks. Input processing, feature engineering and output processing blocks, as shown in **figure 1**.

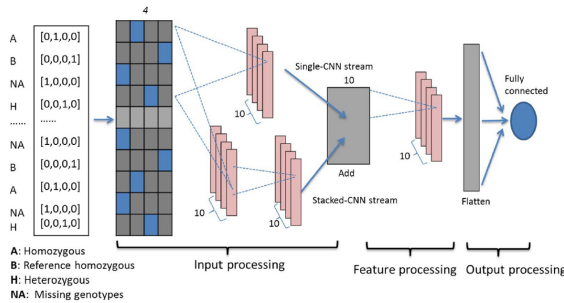


Figure 1: Dual convolutional neural network for phenotype prediction [9]

2.2.1 Input Processing: This module has input layer, a dual-convolutional layer and summation layer.

- **Input layer** processes raw data contains genotypes and genomic markers, and pass it to the next dual convolutional layer.
- **Dual-convolutional layer** contains two stacks of parallel one-dimensional convolution streams. One single convolutional stack contains a convolutional layer with two different kernel sizes of 4 and 20. **Convolutional layer** extracts spatial hierarchies and patterns from input data by applying kernels through a mathematical operation called **convolution**. The dual convolution streams is motivated by the idea of residual learning.

Residual learning is a neural network training technique where layers are designed to learn residual mappings (i.e., the difference between input and output) instead of the direct mapping itself. This technique facilitates training very deep networks by mitigating the vanishing gradient problem and improving convergence. Mathematically, instead of learning a function $H(x)$, the network learns:

$$F(x) = H(x) - x \quad (3)$$

Rearranging:

$$H(x) = F(x) + x \quad (4)$$

where:

- x is the input
- $F(x)$ is the residual function learned by the network
- $H(x)$ is the desired output.

A residual connection is shortcut connection used to directly pass x to the next layer, allowing the network to learn residuals $F(x)$ instead of the full transformation [21].

- **Summation Layer** aggregates results of dual-convolutional layer.

2.2.2 Feature Processing: This block contains a convolutional layer with a kernel size of 4. This convolutional layer further processes genotype features obtained from the input processing block. Aggregate features with different kernel sizes had more powerful representation of genotype features than a single kernel size.

2.2.3 Output Processing: This module contains flatten and dense layers.

- **Flatten layer** takes multi-dimensional features from the previous layer and converts them into a 1D vector, which is then passed to a fully connected (dense) layer.
- **Fully connected layer** contain a single neuron to predict phenotypes.

2.2.4 Activation Function: Dual-stream CNN used **Inverse Square Root Unit (ISRU)** as its activation function.

$$\text{ISRU}(x) = \frac{x}{\sqrt{1 + \alpha x^2}} \quad (5)$$

where α is a hyperparameter that controls the non-linearity of the function. When $\alpha = 0$, ISRU behaves like the identity function. As α increases, the non-linearity becomes more pronounced. ISRU is differentiable and can help mitigate the vanishing gradient problem in deep networks. [3]

2.2.5 Overfitting Control: Deep learning models are prone to overfitting. To reduce the effect of overfitting, the model added dropout layers after convolutional layers with a dropout ratio of 0.75. Then, it applied the L2 regularization to the cost function of mean square error (MSE) between estimated and predicted phenotypes.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (6)$$

This model also monitored the mean absolute error (MAE) on the validation set. It stopped the model training process as soon as the observed MAE stopped decreasing.

2.3 DeepGS convolutional neural network

Wenlong et al. [10] DeepGS model was built using a deep convolutional neural network with architecture 8-32-1, as shown in figure 2. This architecture included one input layer, one convolutional layer (8 neurons), one pooling layer, three dropout layers, two fully connected layers (32 and 1 neuron), and one output layer.

- **Input layer** receives the genotypic markers.
- **Convolutional layer** filters the input matrix with eight kernels that are each 1×18 in size with a stride size of 1×1 to extract feature maps.
- **Max pooling layer** with a stride size of 1×4 reduces the spatial dimensions of feature maps while preserving important features.
- **Fully connected layers** First fully connected layer contains 32 neurons, while second layer contains one neuron. These layers enable complex transformations of the input data into higher-level abstractions.
- **Output layer** This layer predicts phenotype values.

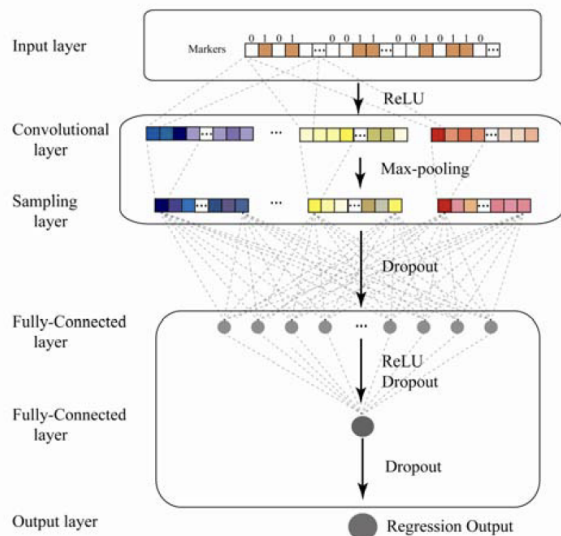


Figure 2: DeepGS model, a deep convolutional neural network for phenotype prediction [10]

2.3.1 Activation Function: Model used rectified linear unit (ReLU) as its activation function, which is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

where:

- x is the input to the activation function,
- $\text{ReLU}(x)$ is the output of the activation function.

2.3.2 Loss Function: The loss function used is the Mean Absolute Error (MAE), which is defined as:

$$\text{MAE} = \frac{1}{m} \sum_{k=1}^m |\hat{y}_k - y_k| \quad (8)$$

where

- m denotes the number of individuals in the training dataset,
- $\hat{y}_k$ is the predicted phenotypic value for the k -th individual,
- y_k is the observed phenotypic value for the k -th individual.

2.3.3 Overfitting Control: This model used dropout layers to avoid overfitting. First dropout layer used dropout value of 0.2, while second dropout layer used dropout 0.05.

2.4 Bayesian Ridge Regression

Silva et al. [5] used BRR for SSR markers identification in Psidium guajava. In this study, we used BRR as a strong statistical baseline compared to CNN models. This model is implemented using scikit-learn. Bayesian ridge regression (BRR) is a form of linear regression where the model coefficients are treated as random variables with a prior distribution.

2.4.1 Model Overview: The Bayesian Ridge Regression model assumes that the relationship between the features X and the target variable y is given by a linear equation:

$$y = X\beta + \epsilon \quad (9)$$

Where:

- $y \in \mathbb{R}^n$ is the vector of observed phenotypic values
- $X \in \mathbb{R}^{n \times p}$ is the matrix of input features (e.g., genotypic data)
- $\beta \in \mathbb{R}^p$ is the vector of regression coefficients
- $\epsilon \sim \mathcal{N}(0, \sigma^2)$ represents the noise term, assumed to be Gaussian with mean 0 and variance σ^2 .

In Bayesian Ridge Regression, instead of finding a point estimate for the coefficients β , we aim to compute their posterior distribution given the data. [4]

Prior distribution on the coefficients: The coefficients β are assumed to follow a Gaussian prior:

$$p(\beta) = \mathcal{N}(\beta | 0, \lambda^{-1}I) \quad (10)$$

where λ is the precision (inverse of variance) parameter of the prior.

Likelihood function: The likelihood of the observed data y given the coefficients β and noise variance σ^2 is:

$$p(y|X, \beta, \sigma^2) = \mathcal{N}(y|X\beta, \sigma^2I) \quad (11)$$

Posterior distribution: Using Bayes' Theorem, the posterior distribution of the coefficients β given the data can be computed as:

$$p(\beta|X, y) \propto p(y|X, \beta, \sigma^2)p(\beta) \quad (12)$$

This leads to a Gaussian posterior distribution with mean and covariance determined by the data.

2.4.2 Loss Function: The optimal coefficients β can be obtained by maximizing the posterior distribution, which is equivalent to minimizing the following loss function (in terms of the log posterior):

$$\mathcal{L}(\beta, \sigma^2) = \frac{1}{2\sigma^2} \|y - X\beta\|^2 + \frac{\lambda}{2} \|\beta\|^2 \quad (13)$$

Where:

- The first term is the least squares loss,
- The second term is the regularization term, controlling the magnitude of the coefficients. [4]

3 Experiments and Results

3.1 Dataset Description:

In this study, soybean dataset from SoyNAM project is used. The goal of Soybean Nested Association Mapping (SoyNAM) project is to improve the yield potential of soybean varieties. SoyNAM dataset contains 25% missing values, makes it good fit to study effects of imputation on deep learning models. The detailed summary of dataset is provided in Table 1.

Table 1: Summary of SoyNAM experimental dataset

Traits	Environment	Sample (N)	Heritability	Reference
Protein	2012 Illinois	5,128	0.545	
Oil	2012 Illinois	5,128	0.617	
Moisture	2012 Illinois	5,128	0.582	Xavier et al.
Height	2013 Illinois	5,138	0.667	[20]

3.2 Data pre-processing:

After acquiring the dataset, data is preprocessed to encode categorical values into binary representations and impute missing values.

One Hot Encoding is used to convert a genotype data matrix into binary representations. Three genotypes 0, 1, 2 represent homozygous, heterozygous, and reference homozygous, and -1 represents missing values encoded into a 4-dimensional binary vector (4 x 1) using sklearn one-hot encoding. For example, after encoding, three genotypes [XX, Xx, xx] are represented as [0, 1, 0, 0], [0, 0, 0, 1], and [0, 0, 1, 0], respectively. The missing genotype is represented as [1, 0, 0, 0]. Encoded genotypes serve as input to phenotype prediction models.

Genetic imputation used to impute missing values in dataset. In this study, **Markov chain haplotyping (MACH)** used to impute the missing values of the soybean dataset.

3.3 Models Training with 10-folds cross validation:

For training, dataset first split into 10 equal folds. Eight folds were used for training the model. One fold used as validation set and the remaining one fold used as a testing set. The same process repeated 10 times, and the average pearson correlation coefficient from the 10 calculations was reported to measure model performance. All three models were trained and tested using same method and evaluated with same evaluation metric. Bayesian ridge regression model trained with imputed data while CNN models trained with both imputed and non imputed data. Originally, DeepGS trained for 6000 epochs and Dual CNN trained for 1000 epochs on Nvidia GTX 1080 Ti GPU for ten minutes. Due to limited resources, in this study both CNN models run for 10 epochs but with 10-folds cross validation and complete training for each trait model between one and a half to two hours.

3.4 Models Performance:

To evaluate models performance, this study used pearson correlation coefficient. **Pearson correlation coefficient** (denoted as r) measures the linear correlation between variables. It ranges from -1 (perfect negative correlation) to +1 (perfect positive correlation), with 0 indicating no linear correlation. The performance of each model is reported in Table 2.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (14)$$

Table 2: Average pearson correlation coefficient for phenotype prediction from 10-folds cross validation

	Height	Moisture	Oil	Protein
Dual-CNN (imp)	0.4175	0.3727	0.4472	0.3872
Dual-CNN (non-imp)	0.5287	0.4452	0.6051	0.5585
DeepGS (imp)	0.0083	0.0309	0.0056	0.0092
DeepGS (non-imp)	0.1041	0.0503	0.0666	0.1402
BRR	0.4617	0.3937	0.4205	0.3981

4 Discussion

4.1 Comparison between CNN and statistical models:

Figure 3, 4 and 5 show the comparison between convolutional neural networks and the bayesian model. On comparing deep learning models such as CNNs with statistical model, such as BRR, results show that dual CNN performs well. Further, results show that deep learning models can train on both imputed and non-imputed data while statistical model needs imputed data. The performance of deep learning models depends on their architecture and parameters as well. For example, Dual CNN used parallel architecture of convolution streams that extracts better features than linear deepGS. Dual CNN performs well with the highest correlation for phenotype prediction compared to deepGS. Due to limited resources we used fewer epochs to train the model so the best performance achieved

by Dual CNN, second best was BRR. DeepGS did not perform well due to fewer epochs and their linear architecture. In literature, to achieve best performance deepGS trained for 6000 epochs whereas I trained these models for 1000 epochs due to unavailability of GPU for this project.

4.2 Effect of genetic imputation on phenotype prediction models:

The effect of imputation studied on convolutional neural networks because these models can train with non-imputed data due to their advanced architecture. **Figure 3 and 4** show the comparison between imputed and non-imputed genotype data. DeepGS and Dual CNN both perform well with non-imputed data compared to imputed data. One reason deep learning model has higher predicting ability on raw datasets may be because the imputation process fills most missing genotypes with reference alleles, and it deflates the effects of different genotypes. Imputation methods assimilate missing genotype effects based on non-missing genotypes, which may compromise the prediction ability of selected quantitative traits.

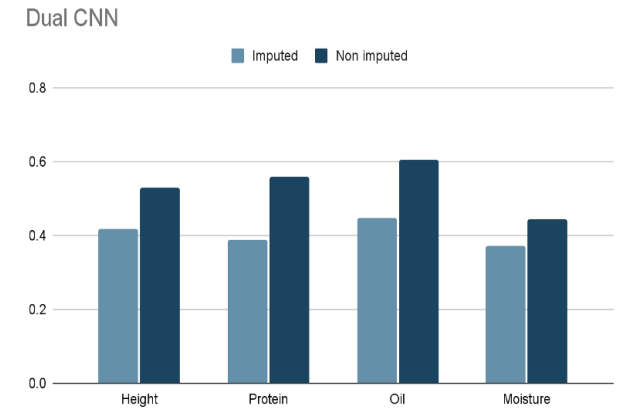


Figure 3: Performance comparison of dual-CNN using imputed and non imputed data.

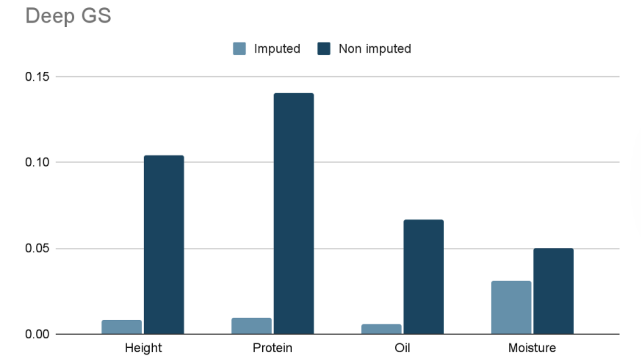


Figure 4: Performance comparison of DeepGS using imputed and non imputed data.

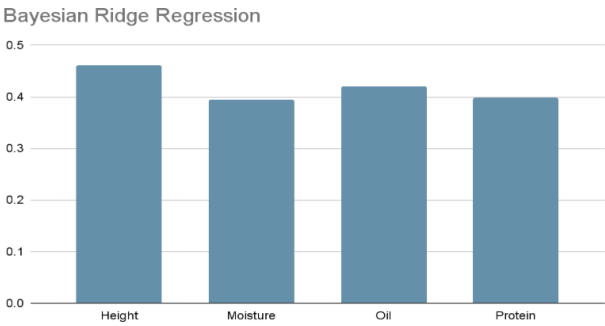


Figure 5: Performance of BRR using imputed data.

4.3 Computational efficiency:

Based on training time, deepGS took very long in training. It maybe due to its linear architecture and model design. Dual CNN used parallel architecture; it took less training time, around 50 minutes compared to deepGS 180 minutes. BRR took 45 minutes training time. SoyNAM dataset has around 5000 samples, it was small dataset. Most of soybean phenotype prediction datasets are small. If the size of the dataset increases for other crops phenotype prediction, model design and architecture play a crucial role in it for convergence. Parallel models like Dual CNN provide good accuracy and converge earlier than linear models. Therefore, it is important to select model design carefully for phenotype prediction with limited resources.

5 Conclusion

This work demonstrates the potential of convolutional neural networks for phenotype prediction in soybean, establishing them as a viable alternative to traditional statistical methods. CNNs excel at capturing local dependencies and gene-gene interactions via their grid-like structure, enabling effective modeling of SNP data without requiring imputation. Our results show that deep learning models, particularly the Dual CNN with its parallel convolutional streams outperform classical approaches even under constraints of missing data.

Future work can explore topological deep learning models such as graph neural networks (GNNs), which may better capture the complex, non-Euclidean relationships among genomic loci that underlie phenotypic traits. Despite promising outcomes, we encountered practical challenges during this study, including limited computational resources and the unavailability of reusable code from prior work. These factors led to extended training and evaluation times. From these challenges we conclude that efficient model design will be critical to advancing deep learning applications in genomics.

References

- [1] Zahra Ahmadi, Farhad Ghafouri-Kesbi, and Pouya Zamani. 2021. Assessing the performance of a novel method for genomic selection: rrBLUP-method6. *Journal of Genetics* 100, 2 (2021), 24.
- [2] Lígia de Oliveira Amaral, Adriano Teodoro Bruzi, Pedro Milanez de Resende, and Karina Barroso Silva. 2019. Pure line selection in a heterogeneous soybean cultivar. *Crop Breeding and Applied Biotechnology* 19, 03 (2019), 277–284.
- [3] Gabor's Milli Research Knowledge Base. 2023. Inverse Square Root Unit (ISRU) Activation Function. https://www.gabormelli.com/RKB/Inverse_Square_Root_

